

**Implementasi Algoritma K-Nearest Neighbors (KNN) dalam Klasifikasi Performa
Akademik Siswa pada Dataset Student Performance**



Baiq Erwina Yolanda	F1D02410039
Fauzan Hari Ramdani	F1D02410047
Lalu Galang Abdullah	F1D02410064
Andre Astamam	F1D02410103

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MATARAM

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Performa akademik siswa merupakan salah satu indikator penting dalam dunia pendidikan. Data siswa biasanya mencakup berbagai aspek seperti nilai, jumlah SKS, latar belakang, serta aktivitas belajar (Tatimu et al., n.d.). Data ini sebenarnya dapat digunakan untuk mengetahui perkembangan dan potensi siswa secara lebih mendalam. Namun, dalam praktiknya, data tersebut sering kali belum dimanfaatkan secara optimal untuk analisis yang lebih lanjut (Setiyadi & Nurdin, 2012). Seiring dengan meningkatnya jumlah data yang tersimpan di institusi pendidikan, proses analisis secara manual menjadi kurang efektif dan memakan waktu. Selain itu, banyaknya atribut dalam data siswa juga membuat proses evaluasi menjadi lebih kompleks, sehingga sulit untuk menemukan pola yang dapat digunakan sebagai dasar pengambilan keputusan (Arrohman et al., n.d.). Untuk mengatasi permasalahan tersebut, diperlukan metode yang mampu mengolah data secara otomatis dan sistematis. Salah satu pendekatan yang dapat digunakan adalah teknik klasifikasi dalam data mining, yang dapat mengelompokkan siswa berdasarkan performa akademiknya (Tatimu et al., n.d.).

Algoritma K-Nearest Neighbors (KNN) merupakan salah satu metode klasifikasi yang sederhana namun cukup efektif (Arrohman et al., n.d.). Algoritma ini bekerja dengan cara menentukan kelas suatu data berdasarkan kedekatannya dengan data lain yang telah diketahui kelasnya. Beberapa penelitian menunjukkan bahwa KNN mampu memberikan hasil klasifikasi yang baik dalam kasus data pendidikan, dengan tingkat akurasi yang cukup tinggi tergantung pada parameter yang digunakan (Sofiyanto et al., n.d.). Selain itu, KNN juga banyak digunakan karena kemudahannya dalam implementasi serta kemampuannya dalam menangani berbagai jenis data. Dalam beberapa penelitian, metode KNN menunjukkan performa yang kompetitif dibandingkan algoritma lain dalam klasifikasi performa siswa (Hidayat et al., 2023).

Namun, ketika jumlah data yang diproses semakin besar, proses perhitungan jarak pada KNN dapat menjadi lebih lambat (Rantika Dewi & Azrai Sirait, 2025). Oleh karena itu, diperlukan pendekatan tambahan untuk meningkatkan efisiensi komputasi. Salah satu solusi yang dapat digunakan adalah komputasi paralel menggunakan Message Passing Interface (MPI), yang memungkinkan proses komputasi dibagi ke dalam beberapa bagian dan dijalankan secara bersamaan sehingga dapat mempercepat waktu eksekusi (Kurniati & Hakim, 2026).

Berdasarkan hal tersebut, penelitian ini dilakukan untuk menerapkan algoritma K-Nearest Neighbors (KNN) dalam mengklasifikasikan performa akademik siswa menggunakan dataset

Student Performance, serta memanfaatkan MPI untuk meningkatkan efisiensi proses komputasi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana cara mengklasifikasikan performa siswa berdasarkan data akademik menggunakan algoritma K-Nearest Neighbor (KNN)?
2. Bagaimana pengaruh penggunaan algoritma KNN terhadap tingkat akurasi dalam klasifikasi data performa siswa?
3. Bagaimana implementasi komputasi paralel menggunakan Message Passing Interface (MPI) dalam proses klasifikasi KNN?
4. Bagaimana perbandingan waktu eksekusi antara proses klasifikasi KNN secara sekuensial dan paralel pada dataset berukuran besar?

1.3 Tujuan

Berdasarkan rumusan masalah yang telah disusun, maka tujuan dari penelitian ini adalah sebagai berikut:

1. Mengimplementasikan algoritma K-Nearest Neighbor (KNN) untuk mengklasifikasikan performa siswa berdasarkan data akademik.
2. Menganalisis tingkat akurasi yang dihasilkan oleh algoritma KNN dalam proses klasifikasi data siswa.
3. Mengimplementasikan metode komputasi paralel menggunakan Message Passing Interface (MPI) pada algoritma KNN.
4. Membandingkan waktu eksekusi antara proses klasifikasi KNN secara sekuensial dan paralel untuk mengetahui tingkat efisiensi yang dihasilkan.

BAB II

DASAR TEORI

A. Data Mining

Data mining merupakan suatu proses penggalian informasi dari sekumpulan data dalam jumlah besar untuk menemukan pola atau hubungan yang sebelumnya tidak diketahui (Suliman, 2021). Proses ini memanfaatkan berbagai teknik dari bidang statistik, machine learning, dan basis data untuk menghasilkan informasi yang berguna dalam pengambilan keputusan. Dalam perkembangannya, data mining menjadi sangat penting karena banyaknya data yang tersedia namun belum dimanfaatkan secara optimal, sehingga diperlukan metode untuk mengolah data tersebut menjadi informasi yang bernilai (Puspita Sari et al., 2023).

Dalam bidang pendidikan, data mining dapat digunakan untuk menganalisis performa siswa berdasarkan berbagai faktor seperti nilai, kehadiran, dan partisipasi dalam kelas (Wardana & Suherman, 2025). Dengan pendekatan ini, pihak sekolah dapat memperoleh gambaran yang lebih objektif mengenai kondisi akademik siswa serta membantu dalam proses evaluasi dan pengambilan keputusan. Pemanfaatan data mining dalam bidang pendidikan terbukti mampu meningkatkan efektivitas analisis data dalam jumlah besar (Indrayati Sijabat et al., 2026).

B. Klasifikasi

Klasifikasi merupakan salah satu teknik dalam data mining yang digunakan untuk mengelompokkan data ke dalam kategori tertentu berdasarkan karakteristik yang dimilikinya (De Wibowo Muhammad Sidik et al., 2020). Proses klasifikasi dilakukan dengan menggunakan data latih yang telah memiliki label, kemudian model yang dihasilkan digunakan untuk memprediksi kelas dari data baru. Teknik ini banyak digunakan dalam berbagai bidang karena kemampuannya dalam memetakan data ke dalam kategori yang jelas dan terstruktur (Homepage & Prasetyo, n.d.-a).

Dalam penelitian ini, metode klasifikasi digunakan untuk menentukan kategori performa siswa berdasarkan data akademik yang tersedia. Klasifikasi dilakukan dengan mengelompokkan siswa ke dalam beberapa kategori nilai, seperti A, B, C, D, dan E. Dengan adanya proses klasifikasi ini, data yang awalnya berupa angka dapat diubah menjadi informasi yang lebih mudah dipahami dan digunakan sebagai dasar pengambilan keputusan (Puspita Sari et al., 2023).

C. Algoritma K-Nearest Neighbor (KNN)

K-Nearest Neighbor (KNN) merupakan salah satu algoritma klasifikasi yang bekerja berdasarkan konsep kedekatan jarak antar data (Abiyan Naufal Hilmi et al., 2024). Algoritma ini mengklasifikasikan suatu data baru dengan cara melihat sejumlah data terdekat (neighbor) dari data tersebut, kemudian menentukan kelas berdasarkan mayoritas dari tetangga terdekat tersebut. Semakin dekat jarak suatu data dengan data lain, maka semakin besar kemungkinan data tersebut berada dalam kelas yang sama (Khairi et al., n.d.).

KNN termasuk dalam metode non-parametrik dan sering disebut sebagai lazy learning, karena tidak melakukan proses pelatihan model secara eksplisit. Proses perhitungan justru dilakukan saat tahap prediksi, yaitu dengan menghitung jarak antara data uji dan seluruh data latih. Meskipun tergolong sederhana, algoritma ini memiliki kemampuan yang cukup baik dalam menghasilkan klasifikasi yang akurat dan telah banyak digunakan dalam berbagai penelitian di bidang data mining (Daulay, 2024a).

Berbagai penelitian telah membuktikan efektivitas KNN dalam berbagai domain, di antaranya pada klasifikasi kelulusan siswa menggunakan dataset *Students Performance* (Arrohman et al., n.d.), prediksi kelulusan siswa sekolah menengah pertama (Puspita Sari et al., 2023), hingga prediksi kualitas air yang juga menghasilkan tingkat akurasi tinggi (Said et al., n.d.). Hasil-hasil tersebut memperkuat posisi KNN sebagai salah satu algoritma klasifikasi yang andal dan dapat diterapkan pada berbagai jenis data, termasuk dalam analisis data akademik pada penelitian ini.

Namun demikian, KNN juga memiliki kelemahan, terutama dalam hal waktu komputasi yang meningkat seiring dengan bertambahnya jumlah data. Hal ini disebabkan karena setiap data uji harus dibandingkan dengan seluruh data latih, sehingga membutuhkan waktu yang lebih lama. Selain itu, performa KNN juga sangat dipengaruhi oleh pemilihan nilai K dan skala data yang digunakan (Daulay, 2024a).

D. Komputasi Paralel

Komputasi paralel merupakan teknik pemrosesan data dengan cara membagi suatu pekerjaan menjadi beberapa bagian yang dapat dijalankan secara bersamaan (Lesmana et al., 2017). Pendekatan ini digunakan untuk meningkatkan efisiensi dan mempercepat waktu eksekusi program, terutama pada pengolahan data dalam jumlah besar. Dengan membagi beban kerja ke beberapa proses, sistem dapat menyelesaikan perhitungan dalam waktu yang lebih singkat dibandingkan dengan metode sekuensial (Said et al., n.d.).

Dalam penelitian ini, komputasi paralel digunakan untuk mengatasi kelemahan algoritma KNN yang membutuhkan waktu komputasi tinggi. Dengan memanfaatkan paralelisme, proses

perhitungan jarak antar data dapat dibagi ke beberapa proses sehingga mempercepat proses klasifikasi secara keseluruhan (Khairi et al., n.d.).

E. Message Passing Interface (MPI)

Message Passing Interface (MPI) merupakan salah satu standar komunikasi yang digunakan dalam pemrograman paralel pada sistem terdistribusi. MPI memungkinkan beberapa proses untuk saling berkomunikasi dan bertukar data selama proses eksekusi berlangsung. Dengan adanya mekanisme komunikasi ini, setiap proses dapat bekerja secara independen namun tetap terkoordinasi dalam menyelesaikan suatu tugas komputasi (Homepage & Prasetyo, n.d.-b) .

Dalam implementasinya, MPI menyediakan berbagai fungsi komunikasi seperti pengiriman data antar proses, pembagian data, serta penggabungan hasil perhitungan. Penggunaan MPI dalam komputasi paralel terbukti mampu meningkatkan efisiensi pemrosesan data, terutama pada aplikasi yang melibatkan perhitungan kompleks dan dataset berukuran besar, sehingga sangat sesuai untuk diterapkan pada algoritma KNN dalam penelitian ini (Daulay, 2024b).

BAB III

METODOLOGI

A. Pengumpulan dan Pemahaman Data

Dataset yang digunakan dalam penelitian ini adalah data performa siswa yang terdiri dari 300.000 data latih dan 75.000 data uji. Dataset ini memiliki beberapa atribut, yaitu *weekly_self_study_hours*, *attendance_percentage*, *class_participation*, dan *total_score* sebagai fitur, serta *grade* sebagai label kelas. Data ini digunakan untuk mengklasifikasikan performa siswa ke dalam beberapa kategori nilai seperti A, B, C, D, dan E. Pemilihan dataset ini didasarkan pada relevansinya terhadap permasalahan klasifikasi dalam bidang pendidikan. Data yang digunakan bersifat numerik sehingga sesuai untuk diolah menggunakan algoritma KNN yang berbasis perhitungan jarak antar data.

B. Pra-pemrosesan Data

Sebelum dilakukan proses klasifikasi, data terlebih dahulu melalui tahap pra-pemrosesan untuk memastikan kualitas data yang digunakan. Tahapan ini meliputi penghapusan atribut yang tidak relevan, seperti *student_id*, karena tidak berpengaruh terhadap proses klasifikasi. Selanjutnya dilakukan normalisasi data menggunakan metode Min-Max Scaling agar setiap fitur memiliki rentang nilai yang sama. Normalisasi ini penting karena algoritma KNN sangat sensitif terhadap perbedaan skala data. Tanpa normalisasi, fitur dengan nilai besar dapat mendominasi perhitungan jarak dan menyebabkan hasil klasifikasi menjadi kurang akurat.

C. Pembagian Data

Dataset yang telah diproses kemudian dibagi menjadi dua bagian, yaitu data latih (*training data*) dan data uji (*testing data*). Pembagian data dilakukan dengan rasio 80:20, di mana 80% data digunakan sebagai data latih dan 20% sisanya digunakan sebagai data uji. Pembagian ini bertujuan untuk melatih model menggunakan sebagian data, kemudian menguji kemampuan model dalam memprediksi data yang belum pernah dilihat sebelumnya. Teknik ini umum digunakan dalam penelitian data mining untuk mengukur performa model secara objektif.

D. Implementasi Algoritma KNN

Pada tahap ini, algoritma K-Nearest Neighbor (KNN) diimplementasikan untuk melakukan proses klasifikasi. Proses dimulai dengan menentukan nilai parameter K sebagai jumlah tetangga terdekat yang digunakan dalam proses klasifikasi. Selanjutnya, dilakukan perhitungan jarak antara data uji dan seluruh data latih menggunakan metode Euclidean Distance. Setelah jarak dihitung, data diurutkan berdasarkan jarak terdekat, kemudian diambil

sejumlah K data terdekat. Kelas yang paling banyak muncul dari tetangga terdekat tersebut akan menjadi hasil prediksi untuk data uji.

E. Implementasi Komputasi Paralel dengan MPI

Untuk meningkatkan efisiensi proses komputasi, algoritma KNN diimplementasikan menggunakan pendekatan komputasi paralel dengan Message Passing Interface (MPI). Dalam implementasi ini, proses utama (rank 0) bertugas membagi data latih ke beberapa proses menggunakan metode *scatter*. Setiap proses kemudian menghitung jarak antara data uji dan bagian data latih yang diterima secara paralel. Hasil perhitungan dari setiap proses dikirim kembali ke proses utama menggunakan metode *gather*. Selanjutnya, proses utama menggabungkan hasil tersebut untuk menentukan tetangga terdekat secara global. Dengan pendekatan ini, proses perhitungan jarak yang sebelumnya dilakukan secara sekuensial dapat dibagi ke beberapa proses sehingga waktu eksekusi menjadi lebih cepat.

F. Proses Prediksi

Setelah model KNN terbentuk, dilakukan proses prediksi terhadap data uji. Setiap data uji akan dihitung jaraknya terhadap data latih, kemudian ditentukan kelasnya berdasarkan mayoritas tetangga terdekat. Pada implementasi paralel, proses prediksi juga dilakukan secara terdistribusi, di mana setiap proses menangani sebagian data uji. Hasil prediksi dari seluruh proses kemudian dikumpulkan untuk membentuk hasil akhir.

G. Evaluasi Model

Evaluasi dilakukan untuk mengukur kinerja model dalam melakukan klasifikasi. Metode evaluasi yang digunakan adalah akurasi, yaitu perbandingan antara jumlah prediksi yang benar dengan total data uji. Selain itu, digunakan juga confusion matrix untuk melihat distribusi hasil klasifikasi secara lebih detail, seperti jumlah data yang diklasifikasikan dengan benar maupun yang salah. Evaluasi ini penting untuk mengetahui seberapa baik model dalam mengenali pola pada data. Selain akurasi, penelitian ini juga mengukur waktu eksekusi program untuk membandingkan performa antara metode sekuensial dan paralel. Waktu eksekusi dihitung menggunakan fungsi waktu pada MPI, sehingga dapat diketahui tingkat efisiensi dari implementasi komputasi paralel.

BAB IV

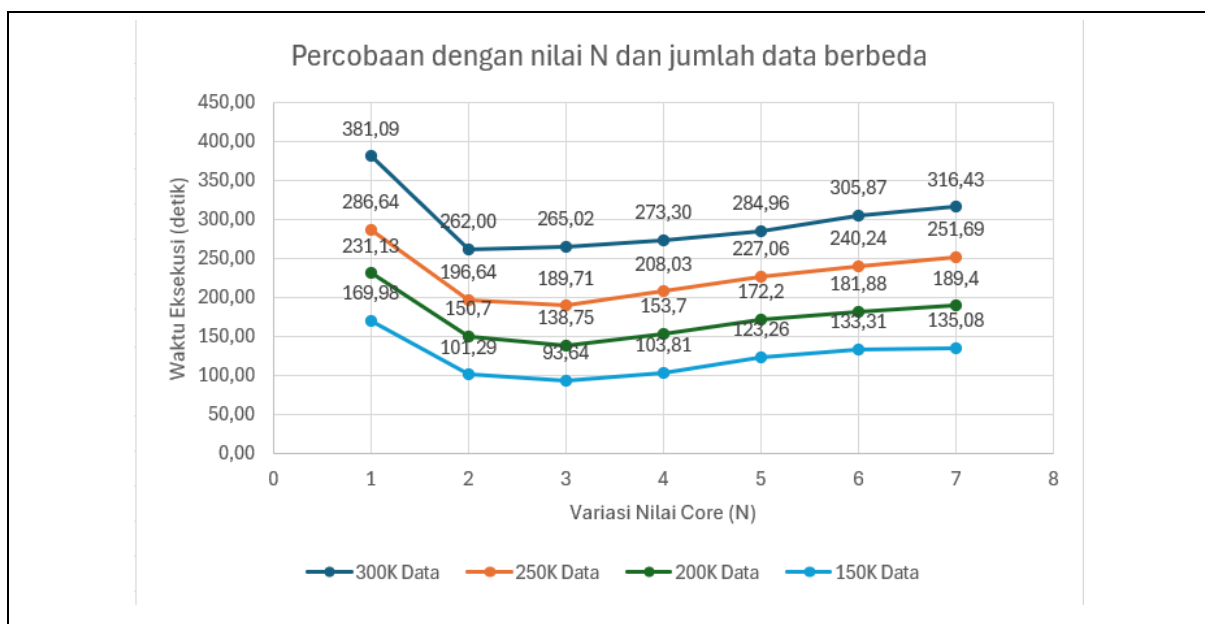
HASIL DAN PEMBAHASAN

A. Menggunakan Jumlah N yang Bervariasi dengan Jumlah Data Berbeda

Tabel 1. Percobaan dengan nilai N dan jumlah data berbeda

N	150.000	200.000	250.000	300.000
1	169,98	231,13	286,64	381,09
2	101,29	150,7	196,64	262,00
3	93,64	138,75	189,71	265,02
4	103,81	153,7	208,03	273,30
5	123,26	172,2	227,06	284,96
6	133,31	181,88	240,24	305,87
7	135,08	189,4	251,69	316,43

Berdasarkan hasil pengujian pada **Tabel 1.**, penurunan waktu eksekusi terjadi secara konsisten saat N ditingkatkan dari 1 hingga 3 (dengan pengecualian pada dataset 300.000 data yang mencapai titik optimal di N=2), di mana pada dataset 300.000 data waktu berkurang dari 381,09 detik menjadi 262,00 detik, pada dataset 250.000 data dari 286,64 detik menjadi 189,71 detik, pada dataset 200.000 data dari 231,13 detik menjadi 138,75 detik, dan pada dataset 150.000 data dari 169,98 detik menjadi 93,64 detik. Namun setelah melewati titik optimal tersebut, waktu eksekusi kembali meningkat pada semua skenario dataset. Pada dataset 300.000 data, waktu naik dari 262,00 detik di N=2 hingga 305,87 detik di N=6, sementara pada dataset 150.000 data meningkat dari 93,64 detik di N=3 hingga 133,31 detik di N=6.



Gambar 1. Grafik Percobaan dengan Jumlah N dan Data Variasi

Berdasarkan grafik garis di atas, dapat diketahui bahwa hubungan antara jumlah *core* (N) dengan waktu eksekusi tidak sepenuhnya berbanding terbalik, melainkan turun lalu naik kembali pada seluruh ukuran dataset yang diuji. Penurunan waktu eksekusi terjadi secara konsisten saat N ditingkatkan dari 1 hingga 3 (dengan pengecualian pada dataset 300.000 data yang mencapai titik optimal di N=2), di mana pada dataset 300.000 data waktu berkurang dari 381,09 detik menjadi 262,00 detik, pada dataset 250.000 data dari 286,64 detik menjadi 189,71 detik, pada dataset 200.000 data dari 231,13 detik menjadi 138,75 detik, dan pada dataset 150.000 data dari 169,98 detik menjadi 93,64 detik. Namun, setelah melewati titik optimal tersebut, waktu eksekusi justru kembali meningkat secara bertahap pada semua skenario dataset. Pada dataset 300.000 data misalnya, waktu naik dari 262,00 detik di N=2 hingga mencapai 316,43 detik di N=7. Pola serupa terjadi pada seluruh ukuran dataset lainnya. Peningkatan waktu ini disebabkan oleh *overhead* komunikasi antar-proses MPI yang semakin besar seiring bertambahnya jumlah *core*, khususnya pada operasi *broadcast* data latih ke seluruh proses.

B. Menggunakan N sebanyak 6 Buah dengan Jumlah Data Latih Bervariasi

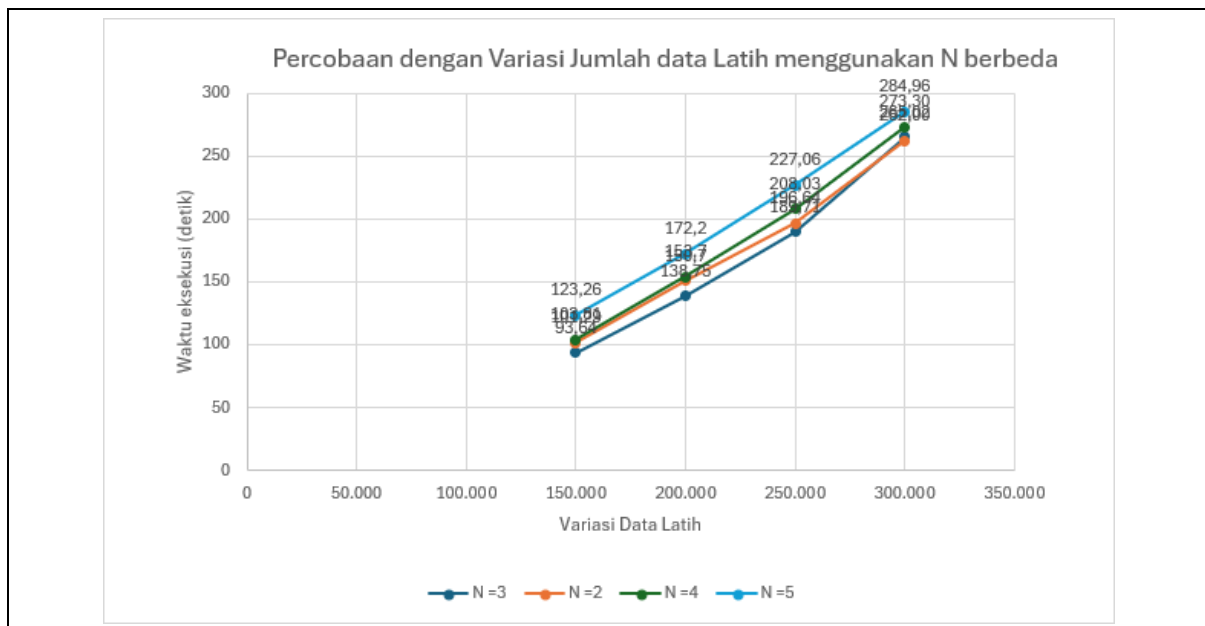
Tabel 2. Hasil Pengujian Variasi Jumlah Data Latih Menggunakan N berbeda

Data Latih	N=2	N=3	N=4	N=5
150.000	101,29	93,64	103,81	123,26
200.000	150,7	138,75	153,7	172,2
250.000	196,64	189,71	208,03	227,06
300.000	262,00	265,02	273,30	284,96

Berdasarkan hasil pengujian pada **Tabel 2.**, terlihat bahwa terdapat hubungan berbanding lurus antara jumlah data latih dengan waktu eksekusi pada seluruh konfigurasi *core* yang diuji. Pada N=3, waktu eksekusi meningkat secara konsisten dari 93,64 detik pada 150.000 data latih hingga 265,02 detik pada 300.000 data latih. Pola yang sama terjadi pada N=2, N=4, dan N=5, di mana waktu eksekusi masing-masing meningkat dari 101,29 detik, 103,81 detik, dan 123,26 detik pada 150.000 data hingga 262,00 detik, 273,30 detik, dan 284,96 detik pada 300.000 data. Hal ini terjadi karena setiap proses MPI tetap harus memproses bagian data yang semakin besar seiring bertambahnya jumlah data latih, sehingga beban komputasi meningkat secara proporsional.

Pada ukuran dataset 150.000 hingga 250.000, N=3 secara konsisten menghasilkan waktu eksekusi terbaik dibandingkan konfigurasi lainnya, sementara N=2 berada di posisi kedua. Namun, pada beban data tertinggi yakni 300.000 data, konfigurasi N=2 berbalik menjadi yang paling efisien dengan waktu 262,00 detik, sedikit mengungguli N=3. Konfigurasi N=4 dan N=5

justru secara keseluruhan lebih lambat dibandingkan N=2 dan N=3 meskipun menggunakan lebih banyak *core*, yang kembali mengonfirmasi adanya *overhead* komunikasi antar-proses yang mendominasi pada jumlah *core* berlebih.



Gambar 2. Grafik Percobaan dengan Jumlah Data Bervariasi

Berdasarkan visualisasi grafik percobaan dengan jumlah data bervariasi di atas, terlihat jelas adanya kenaikan yang konsisten dari awal hingga akhir pengujian pada seluruh konfigurasi *core* yang diuji. Grafik menunjukkan hubungan yang berbanding lurus antara penambahan jumlah data latih dengan lamanya waktu eksekusi program. Pada konfigurasi N=3 yang secara umum merupakan yang paling efisien, waktu eksekusi meningkat dari 93,64 detik pada 150.000 data latih hingga 265,02 detik pada 300.000 data latih. Pola serupa terjadi pada N=2 dari 101,29 detik menjadi 262,00 detik, N=4 dari 103,81 detik menjadi 273,30 detik, dan N=5 dari 123,26 detik menjadi 284,96 detik.

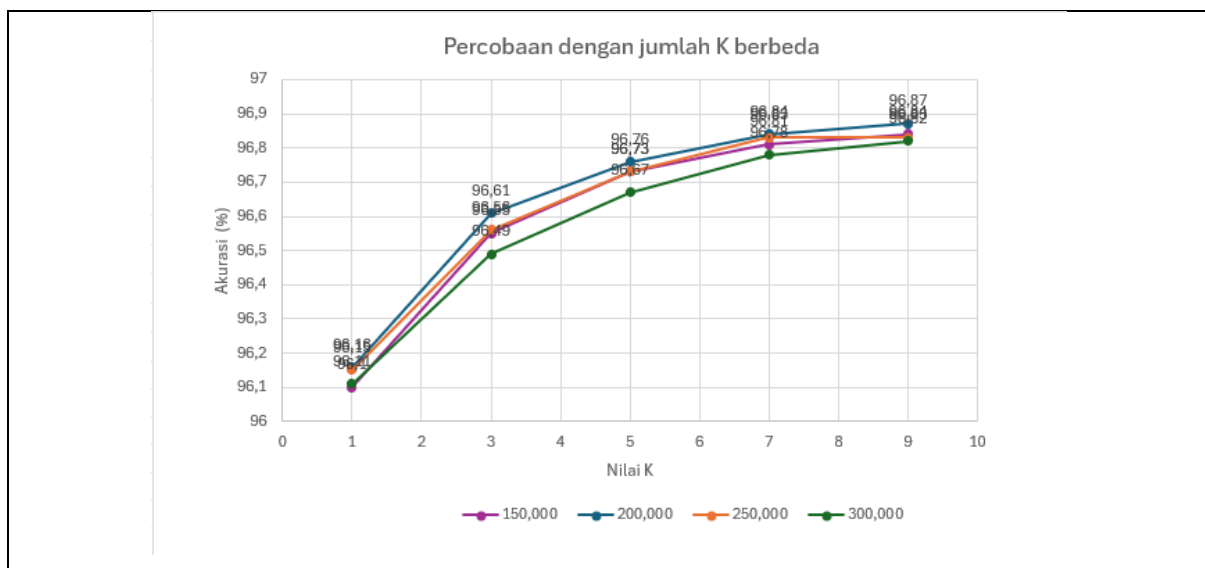
C. Menggunakan N dan Data Tetap dengan Nilai K yang Bervariasi

Tabel 3. Hasil Pengujian Variasi Nilai K (N=3, Data=300.000)

Jumlah Data Latih	K=1	K=3	K=5	K=7	K=9
150.000	96,1%	96,55%	96,73%	96,81%	96,84%
200.000	96,16%	96,61%	96,76%	96,84%	96,87%
250.000	96,15%	96,56%	96,73%	96,83%	96,83%
300.000	96,11%	96,49%	96,67%	96,78%	96,82%

Berdasarkan hasil pengujian pada Tabel 3., terlihat bahwa perubahan nilai K memberikan dampak yang positif pada akurasi klasifikasi model. Mengambil contoh representatif pada penggunaan 300.000 data latih, akurasi terendah tercatat pada K=1 sebesar 96,11%, kemudian meningkat secara konsisten pada K=3 menjadi 96,49%, K=5 sebesar 96,67%, K=7 sebesar 96,78%, hingga mencapai akurasi sebesar 96,82% pada K=9. Berbeda dengan narasi

sebelumnya, akurasi pada pengujian ini tidak menunjukkan fluktuasi naik-turun, melainkan meningkat stabil dan konsisten di setiap penambahan nilai K pada seluruh variasi ukuran dataset. Hal ini menunjukkan bahwa menambah jumlah tetangga yang dijadikan acuan secara konsisten menghasilkan prediksi yang lebih akurat, karena keputusan klasifikasi didasarkan pada lebih banyak titik data di sekitarnya. Jika ditinjau dari keseluruhan tabel, rekor akurasi tertinggi (puncak absolut) sukses dicapai pada pengujian 200.000 data latih dengan $K=9$, yakni sebesar 96,87%. Meskipun demikian, selisih akurasi antara nilai K terkecil dan terbesar (sebagai contoh pada kelompok data 300.000) hanya sebesar 0,71%, yang mengindikasikan bahwa model tetap konsisten dan tidak terlalu sensitif terhadap perubahan nilai K dalam rentang pengujian..



Gambar 6 Percobaan dengan jumlah K bervariasi

Berdasarkan visualisasi grafik di atas, Garis-garis tersebut merupakan visualisasi dari pengujian variasi nilai K terhadap tingkat akurasi model yang sudah dibahas sebelumnya. Sumbu mendatarnya menunjukkan rentang nilai K mulai dari 1 sampai 9, sedangkan sumbu tegaknya menampilkan persentase akurasi (%). Kalau diamati, tren garisnya secara serentak menunjukkan peningkatan (grafik menanjak) pada keempat ukuran dataset. Akurasi terendah berada di titik awal ($K=1$) pada kisaran 96,11%, kemudian meningkat cukup tajam pada $K=3$, dan terus merangkak naik secara landai hingga mencapai titik puncak pada $K=9$ (dengan akurasi absolut tertinggi menyentuh 96,87% pada dataset 200.000). Secara keseluruhan, visualisasi ini semakin mempertegas kesimpulan sebelumnya bahwa memperbesar parameter jumlah tetangga (K) secara konsisten mampu menajamkan akurasi prediksi algoritma, meskipun selisih pertumbuhannya semakin mengecil (stabil) pada nilai K yang lebih besar.

BAB V

KESIMPULAN

Berdasarkan hasil eksperimen dan pembahasan yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Implementasi komputasi paralel dengan MPI terbukti efektif meningkatkan efisiensi waktu eksekusi algoritma KNN dalam klasifikasi performa akademik siswa. Titik optimal paralelisasi pada dataset tertinggi (300.000 data) dicapai pada N=2 *core*, di mana waktu eksekusi berkurang dari 381,09 detik (N=1) menjadi 262,00 detik, menunjukkan percepatan yang signifikan dibandingkan pendekatan sekuensial.
2. Penambahan jumlah *core* melampaui titik optimal justru menurunkan efisiensi akibat *overhead* komunikasi antar-proses MPI yang semakin besar, khususnya pada operasi *broadcast* data latih. Hal ini mencerminkan prinsip *diminishing returns* dalam komputasi paralel, di mana penambahan *core* menjadi N=4 hingga N=7 secara konsisten menghasilkan waktu eksekusi yang lebih lambat dibandingkan titik optimal di N=2 atau N=3.
3. Peningkatan volume data latih berbanding lurus dengan waktu eksekusi pada seluruh konfigurasi *core* yang diuji. Semakin besar data yang diolah, beban komputasi pada setiap proses meningkat secara proporsional, sehingga waktu eksekusi turut meningkat meskipun sistem dijalankan secara paralel. Pada N=3, waktu eksekusi meningkat dari 93,64 detik pada 150.000 data hingga 265,02 detik pada 300.000 data.
4. Variasi nilai K memberikan dampak positif terhadap akurasi namun tidak signifikan secara selisih. Mengambil representasi pada pengujian 300.000 data, akurasi meningkat secara stabil dari 96,11% pada K=1 hingga mencapai titik tertinggi sebesar 96,82% pada K=9 (dengan puncak absolut mencapai 96,87% pada dataset 200.000), dengan selisih total hanya 0,71%. Hal ini menunjukkan bahwa model bersifat konsisten dan tidak terlalu sensitif terhadap perubahan nilai K dalam rentang pengujian yang dilakukan.
5. Kombinasi KNN dan MPI mampu mempertahankan akurasi tinggi sekaligus mereduksi waktu eksekusi, menjadikannya pendekatan yang layak untuk klasifikasi data akademik berskala besar. Untuk penanganan data yang lebih masif di masa mendatang, diperlukan strategi tambahan seperti pembagian partisi data yang lebih seimbang atau peningkatan spesifikasi perangkat keras guna meminimalisir keterbatasan efisiensi pada jumlah *core* tertentu.

LAMPIRAN

A. Source Code

```
from mpi4py import MPI
import pandas as pd
import numpy as np
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

X_latih = None
y_latih = None
X_uji    = None
Y_uji    = None

if rank == 0:
    data_latih = pd.read_csv('data_latih.csv', sep=';', nrows=300000)
    data_uji    = pd.read_csv('data_uji.csv',    sep=';')

    fitur = ['weekly_self_study_hours', 'attendance_percentage',
'class_participation']

    X_latih_raw = data_latih[fitur].values
    y_latih      = data_latih['grade'].values

    X_min = X_latih_raw.min(axis=0)
    X_max = X_latih_raw.max(axis=0)
    X_latih = (X_latih_raw - X_min) / (X_max - X_min)

    X_uji_raw  = data_uji[fitur].values
    X_uji_full = (X_uji_raw - X_min) / (X_max - X_min)
    y_uji_full = data_uji['grade'].values

    # Memecah data menggunakan np.array_split
    X_uji = np.array_split(X_uji_full, size)
    Y_uji = np.array_split(y_uji_full, size)

comm.Barrier()
start_time = time.time()
```

```

# 2. BROADCAST & SCATTER
X_latih = comm.bcast(X_latih, root=0)
y_latih = comm.bcast(y_latih, root=0)

X_uji_lokal = comm.scatter(X_uji, root=0)
y_uji_lokal = comm.scatter(Y_uji, root=0)

# 3. FUNGSI KNN
def knn_klasifikasi(X_latih, y_latih, titik_baru, k):
    # Hitung jarak Euclidean
    jarak = np.sqrt(np.sum((X_latih - titik_baru) ** 2, axis=1))

    # Ambil index dari K tetangga terdekat
    idx_k = np.argpartition(jarak, k)[:k]

    # Label kandidat berdasarkan index
    labels_k = y_latih[idx_k]

    # Menghitung suara dan menentukan label tebakan (Prediksi Final)
    unik, jumlah = np.unique(labels_k, return_counts=True)
    prediksi_final = unik[np.argmax(jumlah)]

    return prediksi_final

# 4. EKSEKUSI PARALEL
K = 9
benar_lokal = 0
jumlah_data_lokal = len(X_uji_lokal)

for i in range(jumlah_data_lokal):
    titik_baru = X_uji_lokal[i]
    label_asli = y_uji_lokal[i]

    tebakan = knn_klasifikasi(X_latih, y_latih, titik_baru, K)

    if tebakan == label_asli:
        benar_lokal += 1

# 5. GATHER
total_benar = comm.reduce(benar_lokal, op=MPI.SUM, root=0)

```

```

total_diuji = comm.reduce(jumlah_data_lokal, op=MPI.SUM, root=0)

if rank == 0:
    end_time = time.time()
    akurasi = (total_benar / total_diuji) * 100
    waktu_eksekusi = end_time - start_time

    print("\nHASIL EVALUASI KNN PARALEL :")
    print(f"Nilai K           : {K}")
    print(f"Core (N)           : {size}")
    print(f"Total Data Latih      : {len(X_latih)}")
    print(f"Total Data Diuji      : {total_diuji}")
    print(f"Total Tebakan Benar   : {total_benar}")
    print(f"Akurasi               : {akurasi:.2f}%")
    print(f"Waktu Eksekusi        : {waktu_eksekusi:.2f} detik")

```

B. Analisis Code

1. Inisialisasi MPI

```

from mpi4py import MPI
import pandas as pd
import numpy as np
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

X_latih = None
y_latih = None
X_uji   = None
Y_uji   = None

```

Potongan kode di atas adalah tahap inisialisasi program paralel. `MPI.COMM_WORLD` adalah komunikator global yang menghubungkan semua proses/rank yang berjalan. `rank` adalah identitas unik tiap proses (0, 1, 2, ... N-1), sedangkan `size` adalah total jumlah proses yang aktif. Variabel `X_latih`, `y_latih`, `X_uji`, `Y_uji` diinisialisasi sebagai `None` di semua rank terlebih dahulu ini penting agar rank selain 0 tidak mengalami `NameError` saat proses `bcast` dan `scatter` dipanggil, karena semua rank menjalankan seluruh baris kode secara bersamaan.

2. Membaca dan Membagi Data (Rank 0)

```

if rank == 0:

```

```

data_latih = pd.read_csv('data_latih.csv', sep=';', nrows=300000)
data_uji    = pd.read_csv('data_uji.csv',    sep=';')

fitur = ['weekly_self_study_hours', 'attendance_percentage',
'class_participation']

X_latih_raw = data_latih[fitur].values
y_latih     = data_latih['grade'].values

X_min = X_latih_raw.min(axis=0)
X_max = X_latih_raw.max(axis=0)
X_latih = (X_latih_raw - X_min) / (X_max - X_min)

X_uji_raw = data_uji[fitur].values
X_uji_full = (X_uji_raw - X_min) / (X_max - X_min)
y_uji_full = data_uji['grade'].values

# Memecah data menggunakan np.array_split
X_uji = np.array_split(X_uji_full, size)
y_uji = np.array_split(y_uji_full, size)

```

Potongan kode di atas hanya dijalankan oleh rank 0, sementara rank lain menunggu. Rank 0 bertugas membaca dua file CSV menggunakan pandas, lalu mendefinisikan daftar kolom fitur(weekly_self_study_hours,attendance_percentage,class_participation) ke dalam variabel fitur agar penulisan kode lebih ringkas, kemudian mengekstrak fitur dan label (grade) dari dataset latih. Setelah itu dilakukan proses normalisasi Min-Max di mana nilai minimum ($X_{\min}$) dan maksimum ($X_{\max}$) dihitung dari data latih untuk menormalisasi X_{latih} ke rentang $[0, 1]$, lalu normalisasi yang sama diterapkan pada data uji ($X_{\text{uji_full}}$) menggunakan skala dari data latih bukan dari data uji itu sendiri untuk mencegah *data leakage* dan menjaga konsistensi transformasi model. Setelah normalisasi, data uji dipotong menjadi beberapa chunk menggunakan $\text{chunk_size} = \text{len}(X_{\text{uji_full}}) // \text{size}$, di mana perulangan `for i in range(size)` memastikan setiap rank mendapat potongannya, dan kondisi `if i == size - 1` memastikan sisa data yang tidak habis dibagi tetap masuk ke rank terakhir sehingga tidak ada data yang terbuang.

3. Broadcast & Scatter

```

comm.Barrier()
start_time = time.time()

```

```
X_latih = comm.bcast(X_latih, root=0)
y_latih = comm.bcast(y_latih, root=0)

X_uji_lokal = comm.scatter(X_uji, root=0)
y_uji_lokal = comm.scatter(Y_uji, root=0)
```

Potongan kode di atas memiliki fungsi `comm.Barrier()` yang dipanggil sebagai titik sinkronisasi agar seluruh *core* memulai proses secara bersamaan, dilanjutkan dengan `start_time = time.time()` untuk memastikan keakuratan pengukuran waktu komputasi tanpa ada *core* yang mencuri *start*. Setelah tersinkronisasi, komunikasi antar-proses MPI mengeksekusi dua operasi utama dengan tujuan berbeda. Fungsi `bcast` (*broadcast*) menyalin seluruh data latih (misalnya 300.000 data) secara utuh dan identik ke semua *rank* sebagai referensi mutlak dalam perhitungan jarak Euclidean KNN. Sebaliknya, fungsi `scatter` memecah dan membagikan array data uji secara merata; sebagai representasi, 75.000 data uji yang dieksekusi pada 3 *core* (N=3) akan didistribusikan sehingga masing-masing *rank* menerima 25.000 data berbeda yang disimpan dalam variabel `X_uji_lokal` dan `y_uji_lokal` untuk diproses secara independen.

4. Fungsi KNN

```
def knn_klasifikasi(X_latih, y_latih, titik_baru, k):
    # Hitung jarak Euclidean
    jarak = np.sqrt(np.sum((X_latih - titik_baru) ** 2, axis=1))

    # Ambil index dari K tetangga terdekat
    idx_k = np.argpartition(jarak, k)[:k]

    # Label kandidat berdasarkan index
    labels_k = y_latih[idx_k]

    # Menghitung suara dan menentukan label tebakan (Prediksi Final)
    unik, jumlah = np.unique(labels_k, return_counts=True)
    prediksi_final = unik[np.argmax(jumlah)]

    return prediksi_final
```

Potongan kode di atas berisi dua fungsi yang menjadi inti algoritma KNN. Fungsi “`def knn_klasifikasi`” menghitung jarak antara satu titik uji dengan satu titik latih menggunakan rumus Euclidean menghitung selisih tiap fitur, mengkuadratkannya, menjumlahkan, lalu diakarkan. Fungsi `knn_klasifikasi` kemudian menggunakan fungsi jarak tersebut untuk membandingkan satu titik uji dengan seluruh data latih, menghasilkan list pasangan (jarak,

label). List ini diurutkan dari jarak terkecil, lalu diambil K tetangga terdekat. Dari K tetangga tersebut dilakukan voting label yang paling banyak muncul menjadi prediksi akhir. Kedua fungsi ini didefinisikan di semua rank karena setiap rank akan memanggilnya secara independen terhadap data uji lokalnya masing-masing.

5. Eksekusi Paralel

```
K = 5
benar_lokal = 0
jumlah_data_lokal = len(X_uji_lokal)

for i in range(jumlah_data_lokal):
    titik_baru = X_uji_lokal[i]
    label_asli = y_uji_lokal[i]

    tebakan = knn_klasifikasi(X_latih, y_latih, titik_baru, K)

    if tebakan == label_asli:
        benar_lokal += 1
```

Potongan kode ini merupakan bagian inti dari proses prediksi KNN. Nilai K ditetapkan sebesar 5, artinya algoritma akan mempertimbangkan 5 tetangga terdekat dalam menentukan kelas suatu data uji. Pada setiap iterasi, program mengambil satu data uji beserta label aslinya, kemudian memanggil fungsi `knn_klasifikasi` yang bekerja dengan menghitung jarak Euclidean terhadap seluruh data latih, lalu menentukan kelas berdasarkan voting mayoritas dari 5 tetangga terdekat. Hasil prediksi kemudian dibandingkan dengan label aslinya jika cocok, variabel `benar_lokal` bertambah 1. Variabel ini nantinya digabungkan dengan hasil dari seluruh proses lain menggunakan operasi `reduce` untuk menghitung akurasi keseluruhan model.

6. Gather dan *Output* Hasil

```
total_benar = comm.reduce(benar_lokal, op=MPI.SUM, root=0)
total_diuji = comm.reduce(jumlah_data_lokal, op=MPI.SUM, root=0)

if rank == 0:
    end_time = time.time()
    akurasi = (total_benar / total_diuji) * 100
    waktu_eksekusi = end_time - start_time

    print("\nHASIL EVALUASI KNN PARALEL :")
    print(f"Nilai K                : {K}")
    print(f"Core (N)                  : {size}")
```

```
print(f"Total Data Latih      : {len(X_latih)}")
print(f"Total Data Diuji      : {total_diuji}")
print(f"Total Tebakan Benar   : {total_benar}")
print(f"Akurasi                : {akurasi:.2f}%")
print(f"Waktu Eksekusi         : {waktu_eksekusi:.2f} detik")
```

Potongan kode ini merupakan tahap pengumpulan hasil dan pelaporan akhir. Operasi `comm.reduce` dengan `MPI.SUM` digunakan untuk mengumpulkan dan menjumlahkan nilai `benar_lokal` dari seluruh proses menjadi `total_benar` di rank 0, hal yang sama juga dilakukan pada `jumlah_data_lokal` untuk mendapatkan total keseluruhan data yang diuji. Setelah semua hasil terkumpul, hanya rank 0 yang melanjutkan eksekusi untuk mencatat waktu akhir, menghitung akurasi dengan membagi `total_benar` dengan `total_diuji` dikalikan 100, serta menghitung waktu eksekusi keseluruhan. Rank 0 kemudian mencetak seluruh hasil evaluasi meliputi nilai K, jumlah core, total data latih, total data diuji, jumlah prediksi benar, akurasi, dan waktu eksekusi. Hanya rank 0 yang bertugas mencetak hasil karena dialah satu-satunya proses yang memiliki data lengkap hasil akumulasi dari seluruh rank.

DAFTAR PUSTAKA

- Abiyan Naufal Hilmi, Eva Yulia Puspaningrum, & Henni Endah Wahanani. (2024). Implementasi Algoritma K-Nearest Neighbor (KNN) untuk Identifikasi Penyakit pada Tanaman Jeruk Berdasarkan Citra Daun. *Router : Jurnal Teknik Informatika Dan Terapan*, 2(2), 107–117. <https://doi.org/10.62951/router.v2i2.78>
- Arrohman, M. R., Khoiriyah, N., Fawaida, A., & Fithri, D. L. (n.d.). *ANALISIS KLASIFIKASI KELULUSAN SISWA MENGGUNAKAN METODE KNN (K-NEAREST NEIGHBOR) RANDOM FOREST PADA DATASET “STUDENTS PERFORMANCE.”*
- Daulay, R. S. (2024a). Analisis Kritis dan Pengembangan Algoritma K-Nearest Neighbor (KNN): Sebuah Tinjauan Literatur. *Jurnal Pendidikan Sains Dan Komputer*, 4(02), 131–141. <https://doi.org/10.47709/jpsk.v4i02.5055>
- De Wibowo Muhammad Sidik, A., Himawan Kusumah, I., Suryana, A., Artiyasa, M., & Pradiftha Junfithrana, A. (2020). *Gambaran Umum Metode Klasifikasi Data Mining*. 2(2), 34–38.
- Hidayat, T., Handayani, Y., & Syaifudin, A. (2023). IMPLEMENTASI ALGORITMA K-NEAREST NEIGHBOR. *Jurnal Ilmiah Rekayasa Dan Manajemen Sistem Informasi*, 9(2), 118–124.
- Homepage, J., & Prasetyo, B. (n.d.-a). *IJIRSE: Indonesian Journal of Informatic Research and Software Engineering Implementation of K-Nearest Neighbor Classification Algorithm as Decision Support Method in Staple Food Aid Distribution to Society Implementasi Algoritma Klasifikasi K-Nearest Neighbor Sebagai Metode Pendukung Keputusan Dalam Distribusi Bantuan Sembako Pada Masyarakat.*
- Indrayati Sijabat, P., Br Barus, E., Laora SLingga, Y., & Amelia Putri, D. (2026). Penerapan Algoritma K-Nearest Neighbor (KNN) Untuk Menganalisis Pendapat Siswa Terhadap Edukasi Anti Perundungan. *JURNAL SISTEM INFORMASI TGD*, 5(1), 217–222. <https://ojs.trigunadharma.ac.id/index.php/jsi>
- Khairi, A., Fahrezi, I., Sahputra, I., & Anshari, F. (n.d.). *Implementation of K-Nearest Neighbor (KNN) Method to Determine House Locations in Batuphat and Tambon Tunong Areas, Aceh The Application of the K-Nearest Neighbor (KNN) Method to Determine House Locations in the Batuphat and Tambon Tunong Areas, Aceh.* <https://doi.org/10.24002/jbi.v10i1.184>

- Kurniati, A., & Hakim, A. (2026). Analisis kemiskinan multidimensi di Kota Magelang dengan metode Multidimensional Poverty Index (MPI). *Jurnal Kebijakan Ekonomi Dan Keuangan*, 237–246. <https://doi.org/10.20885/JKEK.vol4.iss2.art11>
- Lesmana, A., Magister, W., Elektro, T., Angela, M., Magister, K., & Mulyanti, S. (2017). *KOMPUTASI PARALEL UNTUK PENGOLAHAN PRESTASI AKADEMIK MAHASISWA*.
- Puspita Sari, D., Shofia Hilabi, S., & Agustia Hananto. (2023). Penerapan Data Mining Metode K-Nearest Neighbor Untuk Memprediksi Kelulusan Siswa Sekolah Menengah Pertama. *SMARTICS Journal*, 9(1), 14–19. <https://doi.org/10.21067/smartics.v9i1.8088>
- Rantika Dewi, & Azrai Sirait. (2025). Implementasi Algoritma K-Nearest Neighbor (KNN) untuk Klasifikasi Tingkat Literasi Siswa. *Jurnal Teknik Informatika Dan Teknologi Informasi*, 5(3), 56–70. <https://doi.org/10.55606/jutiti.v5i3.6086>
- Said, H., Matondang, N., Nurramdhani Irmanda, H., & Informasi, S. (n.d.). *Penerapan Algoritma K-Nearest Neighbor Untuk Memprediksi Kualitas Air Yang Dapat Dikonsumsi Application of K-Nearest Neighbor Algorithm to Predict Consumable Water Quality* (Vol. 21, Number 2). Retrieved www.kaggle.com
- Setiyadi, D., & Nurdin, A. (2012). *DATA MINING POTENSI AKADEMIK SISWA BERBASIS ONLINE* (Vol. 2, Number 1).
- Sofiyanto, I. N., Saputra, A. B., Rakhmadianto, R. F., Nur, H., Febrianto, E., Endiwan, A. M., & Sari, A. A. (n.d.). Prediksi Kelulusan Mahasiswa Menggunakan Algoritma K-Nearest Neighbor (KNN) dengan Klasifikasi Biner. *Seminar Nasional Teknologi Informasi Dan Bisnis (SENATIB)*, 2025.
- Suliman, S. (2021). Implementasi Data Mining Terhadap Prestasi Belajar Mahasiswa Berdasarkan Pergaulan dan Sosial Ekonomi Dengan Algoritma K-Means Clustering. *SIMKOM*, 6(1), 1–11. <https://doi.org/10.51717/simkom.v6i1.48>
- Tatimu, F., Palilingan, V. R., & Rianto, I. (n.d.). Analisis Klasifikasi Kelulusan Mahasiswa menggunakan Algoritma K-Nearest Neighbor di Fakultas Teknik Universitas Negeri Manado. In *JOURNAL OF EDUCATION METHOD AND TECHNOLOGY* (Vol. 3).
- Wardana, M. A., & Suherman, S. (2025). Penerapan Data Mining Pengelompokan Data Penjualan Motor di PT. Nusantara Surya Sakti Soppeng Menggunakan Metode K-Means

Clustering. *Jurnal Ilmiah Sistem Informasi Dan Teknik Informatika (JISTI)*, 8(1), 123–132. <https://doi.org/10.57093/jisti.v8i1.282>